

Machine Learning and Data Mining

Jeremy Le — UNSW COMP9417 25T2

Contents

1	Regression	3
1.1	Introduction to Regression	3
1.2	Covariance and Correlation	3
1.3	Multivariate Regression	3
1.4	Optimization by Gradient Descent	4
1.4.1	Batch Mode Gradient Descent	4
1.4.2	Incremental mode (Stochastic) Gradient Descent	5
1.5	Regularisation	5
1.5.1	Regularised Multivariate Least-Squares Regression	5
1.6	Bias-Variance Decomposition	5
2	Classification	6
2.1	Introduction	6
2.2	Evaluating Classification	6
2.3	Perceptron	6
2.4	Logistic Regression	7
2.5	Bayes Theorem and Maximum Likelihood	8
3	Nonparametric Modelling	8
3.1	Distances	8
3.2	Nearest Neighbour	9
3.2.1	k -Nearest Neighbour (k NN) Algorithm	9
3.3	Tree Learning	10
3.3.1	Top-Down Induction of Decision Trees (TDIDT)	10
3.3.2	Information Gain	10
3.3.3	Overfitting	10
3.3.4	Regression Trees	10
3.3.5	Model Trees	11
4	Kernel Methods	11
4.1	Introduction	11
4.2	Dual Perceptron	12
4.3	Support Vector Machines	12
4.4	The Kernel Trick	12
5	Ensemble Learning	13
5.1	Bias and Variance	13
5.2	Stability	14
5.3	Ensemble Methods	14

5.3.1	Bagging	14
5.3.2	Random Forest	15
5.3.3	Boosting	15
5.3.4	Other Ensemble Learning	15
6	Neural Learning	16
6.1	The Multi-layer Perceptron	16
6.2	Back-Propagation	16
7	Unsupervised Learning	17
7.1	Introduction	17
7.2	Clustering	17
7.2.1	k -means Clustering	17
7.3	Hierarchical Clustering	18
7.3.1	DBSCAN	18
7.4	Principal Component Analysis	19
8	Learning Theory	19
8.1	No Free Lunch Theorem	19
8.2	A Conservation Theorem of Generalisation Performance	19
8.3	PAC Learning	19
8.4	Vapnik-Chervonenkis (VC) Dimension	20
8.5	Mistake Bounds	20
8.5.1	WINNOW	20
8.6	The WEIGHTED MAJORITY Algorithm	20

1 Regression

1.1 Introduction to Regression

A regression equation is a simple model of numerical prediction. This models an output y as a function of input x . Given a set of data:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

If there is a linear relationship, simple linear regression should be a suitable model:

$$\hat{y} = a + bx$$

For each x_i in the data, we have a corresponding y_i , and a corresponding prediction $\hat{y}_i = a + bx_i$. The difference $e_i = y_i - \hat{y}_i$ between the actual and predicted values of our model tells us something about how well it fits the data.

1.2 Covariance and Correlation

The correlation coefficient is a number between -1 and +1 that indicates whether a pair of variables X and Y are associated or not, and whether the scatter in the association is high or low.

Only appropriate when X and Y are roughly linearly associated.

Correlation between X and Y as a population quantity is

$$r = \frac{\text{cov}(X, Y)}{\sqrt{\text{var}(X)}\sqrt{\text{var}(Y)}}$$

where given data $((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$,

$$\text{var}(X) = \frac{1}{n-1} \sum (x_i - \bar{x})^2, \quad \text{var}(Y) = \frac{1}{n-1} \sum (y_i - \bar{y})^2, \quad \text{cov}(X, Y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y}).$$

The sign of r is a quick way to checking whether there is some linear association, and the sign gives you the direction of the association. However correlation coefficient does not model any relationship.

1.3 Multivariate Regression

Data has p numeric features, and we need numeric prediction $\implies$ regression.

Linear models, i.e., outcomes is *linear* combinations of attributes

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_px_p$$

Predicted value for first training instance $\mathbf{x}^{(1)}$ is:

$$b_0x_0^{(1)} + b_1x_1^{(1)} + \dots + b_px_p^{(1)} = \sum_{i=0}^p b_ix_i^{(1)}$$

$p + 1$ weights or coefficients must be learned on training data. We have $x_0^{(1)} = 1$.

Difference between predicted and actual values is the error. $p + 1$ coefficients are chosen so that mean of sum of squared errors on all instances in training data is minimized.

Mean Squared Error (MSE):

$$\frac{1}{n} \sum_{j=1}^n \left(y^{(j)} - \sum_{i=0}^p b_i x_i^{(j)} \right)^2$$

The solution for the model parameters θ under univariate linear regression is:

$$\hat{b} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

1.4 Optimization by Gradient Descent

We want to

$$\underset{\mathbf{x}}{\text{maximise}} f(\mathbf{x}) \text{ subject to } \mathbf{x} \in \mathcal{X}$$

which is equivalent to

$$\underset{\mathbf{x}}{\text{minimise}} -f(\mathbf{x}) \text{ subject to } \mathbf{x} \in \mathcal{X}$$

- The problem is to find some $\mathbf{x}$ which is called a design point, which is a p -dimensional vector of values for p design variables.
- These values are manipulated by an optimization algorithm to find a minimizer, a solution $\mathbf{x}^*$ that minimizes the objective function f .
- A solution must be an element of the feasible set $\mathcal{X}$, and may be subject to additional constraints called constrained optimization.

A general iterative algorithm to optimise some function f :

1. start with initial point $\mathbf{x} = \mathbf{x}_0$
2. select a search direction $\mathbf{g}$, usually to decrease $f(\mathbf{x})$
3. select a step length α
4. set $\mathbf{s} = \alpha \mathbf{g}$
5. set $\mathbf{x} = \mathbf{x} + \mathbf{s}$
6. go to step 2, unless convergence criteria are met

1.4.1 Batch Mode Gradient Descent

Do until termination condition is satisfied

- Compute the gradient $\nabla E_D[\mathbf{w}]$
- $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_D[\mathbf{w}]$

1.4.2 Incremental mode (Stochastic) Gradient Descent

Do until satisfied

- For each training example d in D
 - Compute the gradient $\nabla E_d[\mathbf{w}]$
 - $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_D[\mathbf{w}]$

1.5 Regularisation

Regularisation is a general method to avoid overfitting by applying additional constraints to the weight vector. A common approach is to make sure the weights are, on average, small in magnitude: this is referred to as shrinkage.

For example, MSE as a loss function given data $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\theta}(\mathbf{x}_i))^2$$

and with a penalty function:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\theta}(\mathbf{x}_i))^2 + \lambda \sum_j \theta_j^2$$

or:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\theta}(\mathbf{x}_i))^2 + \lambda \sum_j |\theta_j|$$

1.5.1 Regularised Multivariate Least-Squares Regression

The multivariate least-squares regression problem is often written as an optimisation problem using vector notation:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}\mathbf{w})^{\top} (\mathbf{y} - \mathbf{X}\mathbf{w})$$

where $\mathbf{w}$ represents the parameters as “weights”. The regularised versions of this optimisation is then as follows:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}\mathbf{w})^{\top} (\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \|\mathbf{w}\|^2$$

where $\|\mathbf{w}\|^2 = \sum_i w_i^2$ is the squared norm of the parameters or weights vector $\mathbf{w}$, or, equivalently, the dot product $\mathbf{w}^{\top} \mathbf{w}$; λ a scalar determining the amount of regularisation.

This regularisation problem still has a closed-form solution:

$$\hat{\mathbf{w}} = (\mathbf{X}^{\top} \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^{\top} \mathbf{y}$$

where $\mathbf{I}$ denotes the identity matrix.

1.6 Bias-Variance Decomposition

It can be shown that

$$\text{MSE}(\theta) := \text{var}(\theta) + \text{bias}(\theta)^2.$$

2 Classification

2.1 Introduction

The idea is to learn a classifier, which is usually a function mapping from an input data point to one of a set of discrete outputs, i.e., the classes.

- Does the algorithm require all training data to be present before the start of learning? If yes, then it is categorised as **batch learning** algorithm.
- If however, it can continue to learn a new data arrives, it is an **online learning** algorithm.
- If the model has a fixed number of parameters it is categorised as **parametric**.
- Otherwise if the number of parameters grows as part of training it is categorised as **non-parametric**.

2.2 Evaluating Classification

Classification accuracy on a sample of labelled pairs $(x, c(x))$ given a learned classification model that predicts, for each instance x , a class value $\hat{c}(x)$:

$$\text{acc} = \frac{1}{|\text{Test}|} \sum_{x \in \text{Test}} I[\hat{c}(x) = c(x)]$$

where Test is a test set and $I[\cdot]$ is the indicator function which is 1 if and only if its argument evaluates to true, and 0 otherwise. Classification error is $1 - \text{acc}$.

For the two-class prediction case:

- Two kinds of error (incorrect classification):
 - False Positive and False Negative may have different costs.
- Two kinds of correct classification:
 - True Positive and True Negative may have different “benefits”.
- Note: total number of test set examples $N = TP + FN + FP + TN$

2.3 Perceptron

Output o is thresholded sum of products of inputs and their weights:

$$o(x_1, \dots, x_n) = \begin{cases} +1 & \text{if } w_0 + w_1x_1 + \dots + w_nx_n > 0 \\ -1 & \text{otherwise.} \end{cases}$$

Or in vector notation:

$$o(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} > 0 \\ -1 & \text{otherwise.} \end{cases}$$

Key idea: Learning is “finding a good set of weights”. Perceptron learning is simply an iterative weight-update scheme:

$$w_i \leftarrow w_i + \Delta w_i$$

where the weight update Δw_i depends only on misclassified examples and is modulated by a “smoothing” parameter η typically referred to as the “learning rate”.

- We combine the misclassified positive and negative cases with a single update rule:

$$\mathbf{w}' = \mathbf{w} + \eta y_i \mathbf{x}_i$$

- Here y_i acts to change the sign of the update, corresponding to whether a positive or negative example was misclassified.

The algorithm just iterates over the training examples applying the weight update rule until all the examples are correctly classified.

Algorithm Perceptron(D, η) // perceptron training for linear classification

Input: labelled training data D in homogeneous coordinates; learning rate η .

Output: weight vector $\mathbf{w}$ defining classifier $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x})$.

```

1  $\mathbf{w} \leftarrow \mathbf{0}$  // Other initialisations of the weight vector are possible
2  $converged \leftarrow \text{false}$ 
3 while  $converged = \text{false}$  do
4    $converged \leftarrow \text{true}$ 
5   for  $i = 1$  to  $|D|$  do
6     if  $y_i \mathbf{w} \cdot \mathbf{x}_i \leq 0$  then // i.e.,  $\hat{y}_i \neq y_i$ 
7        $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$ 
8        $converged \leftarrow \text{false}$  // We changed  $\mathbf{w}$  so haven't converged yet
9     end
10  end
11 end
```

Perceptron training will converge (under some mild assumptions) for linearly separable classification problems. A labelled data set is linearly separable if there is a linear decision boundary that separates the classes.

2.4 Logistic Regression

In the case of a two-class classification problem, if we model the probability $P(Y = 1)$ of an instance $\mathbf{x}$ being a positive example like this:

$$P(Y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

then this probability vs. the alternative $(1 - P(Y = 1))$ can be written like this:

$$\ln \frac{P(Y = 1 | \mathbf{x})}{1 - P(Y = 1 | \mathbf{x})} = \mathbf{w}^\top \mathbf{x}$$

The quantity on the LHS is called the logit and we are defining a linear model for the logit.

An iterative gradient ascent method can be used to maximise log likelihood. The (conditional) log likelihood is

$$\sum_{i=1}^N y^{(i)} \log P(1 | \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - P(1 | \mathbf{x}^{(i)})).$$

Over a set of N examples, choose $\mathbf{w}$ to maximise this expression, noting that $y^{(i)}$ is always either 0 or 1.

2.5 Bayes Theorem and Maximum Likelihood

Bayes Theorem for estimating probability of model m from data D :

$$P(m | D) = \frac{P(D | m)P(m)}{P(D)}$$

where

$P(m)$ = prior probability of model m

$P(D)$ = prior probability of training data D

$P(m | D)$ = probability of m given D

$P(D | m)$ = probability of D given m

Maximum a posteriori hypothesis m_{MAP} :

$$\begin{aligned} m_{MAP} &= \arg \max_{m \in \mathcal{M}} P(m | D) \\ &= \arg \max_{m \in \mathcal{M}} \frac{P(D | m)P(m)}{P(D)} \\ &= \arg \max_{m \in \mathcal{M}} P(D | m)P(m) \end{aligned}$$

If assume $P(m_i) = P(m_j)$ then can further simplify, and choose the Maximum Likelihood (ML) hypothesis

$$h_{ML} = \arg \max_{m_i \in \mathcal{M}} P(D | m_i)$$

This assumption means we believe all models are a priori equally likely.

Bayes optimal classification:

$$\arg \max_{y_i \in \mathcal{Y}} \sum_{m_i \in \mathcal{M}} P(y_j | m_i)P(m_i | D)$$

Problem: Bayes optimal classification is typically intractable and is therefore not used in practice!

3 Nonparametric Modelling

3.1 Distances

If $\mathcal{X} = \mathbb{R}^d$, the Minkowski distance of order $p > 0$ between $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ is defined as:

$$\text{Dis}_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^d |x_j - y_j|^p \right)^{1/p} = \|\mathbf{x} - \mathbf{y}\|_p$$

where, for some $\mathbf{z} \in \mathcal{X}$, $\|\mathbf{z}\|_p = \left(\sum_{j=1}^d |z_j|^p \right)^{1/p}$ is the p -norm of the vector $\mathbf{z}$.

Theorem: The arithmetic mean minimises squared Euclidean distance. The arithmetic mean μ of a set of data points D in a Euclidean space is the unique point that minimises the sum of squared Euclidean distances to those data points.

3.2 Nearest Neighbour

Stores all training examples $(x_i, f(x_i))$.

Nearest neighbour:

- Given query instance x_q , first locate nearest training example x_n , then estimate $\hat{f}(x_q) \leftarrow f(x_n)$.

k -Nearest neighbour:

- Given x_q , take vote among its k nearest neighbours (if discrete-valued target function)
- take mean of f values of k nearest neighbours (if real-valued)

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^k f(x_i)}{k}$$

3.2.1 k -Nearest Neighbour (k NN) Algorithm

Training algorithm:

- For each training example $(x_i, f(x_i))$, add the example to the list *training_examples*.

Classification algorithm:

- Given a query instance x_q to be classified,
 - Let $x_1 \dots x_k$ be the k instances from *training_examples* that are nearest to x_q by the distance function
 - Return

$$\hat{f}(x_q) \leftarrow \arg \max_{y \in \mathcal{Y}} \sum_{i=1}^k I[y, f(x_i)]$$

where $I[a, b] = 1$ if $a = b$ and 0 otherwise.

Use Nearest Neighbour when

- Instances map to points in $\mathbb{R}^d$
- Low-dimensional data, say less than 20 features per instance (Curse of Dimensionality)
- Lots of training data
- Target function (classification or regression) is “local”
- No requirement for “explanatory” model to be learned
- Small k has low bias, high variance. Increasing k increases bias and decreases variance.

3.3 Tree Learning

3.3.1 Top-Down Induction of Decision Trees (TDIDT)

1. Select A as the “best” decision attribute for the next *node*
2. Assign A as decision attribute for *node*
3. For each value of A , create new descendant of *node*
4. Sort training examples to leaf nodes
5. If training examples perfectly classified, then STOP, else iterate over new leaf nodes

3.3.2 Information Gain

Entropy measures the “impurity” of S . $Entropy(S)$ = expected number of bits needed to encode class ($\oplus$ or $\ominus$) of randomly drawn member of S (under the optimal, shortest-length code).

$$Entropy(S) = -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$$

$Gain(S, A)$ = expected reduction in entropy due to sorting on A .

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

3.3.3 Overfitting

To avoid overfitting in Tree learning there are two main approaches

- **pre-pruning**: stop growing when further data splits are not useful
- **post-pruning** grow full tree, then remove sub-trees which may be overfitting

Post-pruning more common:

- can prune using a pessimistic estimate of error
 - measure training set error
 - adjust by a factor dependent on “confidence” hyperparameter
- or can use cross-validation to estimate error during pruning

3.3.4 Regression Trees

Difference to decision trees:

- Splitting criterion: minimising intra-subset variation
- Pruning criterion: based on numeric error measure
- Leaf node predicts average class values of training instances reaching that node

Can approximate piecewise constant functions, easy to interpret, more sophisticated version: model trees.

3.3.5 Model Trees

- Like regression trees but with linear regression functions at each node
- Linear regression applied to instances that reach a node after full tree has been built
- Only a subset of the attributes is used for linear regression. Attributes occurring in subtree (+ maybe attributes occurring in path to the root)
- Fast: overhead for linear regression not large because usually only a small subset of attributes is used in tree

4 Kernel Methods

4.1 Introduction

A **classifier** is a mapping $\hat{c} : \mathcal{X} \rightarrow \mathcal{C}$, where $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ is a finite and usually small set of **class labels**. We use the ‘hat’ to indicate that $\hat{c}(x)$ is an estimate of the true but unknown function $c(x)$.

A **scoring classifier** is a mapping $\hat{\mathbf{s}} : \mathcal{X} \rightarrow \mathbb{R}^k$, i.e., a mapping from the instance space to a k -vector of real numbers. The boldface notation indicates that a vector $\hat{\mathbf{s}}(x) = (\hat{s}_1(x), \dots, \hat{s}_k(x))$ rather than a single number; $\hat{s}_i(x)$ is the score assigned to class C_i for instance x .

If we take the true class $c(x)$ as $+1$ for positive examples and -1 for negative examples, then the quantity $z(x) = c(x)\hat{s}(x)$ is positive for correct predictions and negative for incorrect predictions: this quantity is called the **margin** assigned by the scoring classifier to the example.

We would like to reward large positive margins, and penalise large negative values. This is achieved by a means of a so-called **loss function** $L : \mathbb{R} \rightarrow [0, \infty)$ which maps each example’s margin $z(x)$ to an associated loss $L(z(x))$.

4.2 Dual Perceptron

Algorithm DualPerceptron(D) // perceptron training in dual form

Input: labelled training data D in homogeneous coordinates

Output: coefficients α_i defining weight vector $\mathbf{w} = \sum_{i=1}^{|D|} \alpha_i y_i \mathbf{x}_i$

```
1  $\alpha_i \leftarrow 0$  for  $1 \leq i \leq |D|$ 
2  $converged \leftarrow \text{false}$ 
3 while  $converged = \text{false}$  do
4    $converged \leftarrow \text{true}$ 
5   for  $i = 1$  to  $|D|$  do
6     if  $y_i \sum_{j=1}^{|D|} \alpha_j y_j \mathbf{x}_i \cdot \mathbf{x}_j \leq 0$  then
7        $\alpha_i \leftarrow \alpha_i + 1$ 
8        $converged \leftarrow \text{false}$ 
9     end
10  end
11 end
```

4.3 Support Vector Machines

A bunch of math, look at the lecture slides. TLDR is finds optimal hyperplanes which best separates the data into different classes by maximising the margin between them.

4.4 The Kernel Trick

Let $\mathbf{x}_1 = (x_1, y_1)$ and $\mathbf{x}_2 = (x_2, y_2)$ be two data points, and consider the mapping $(x, y) \mapsto (x^2, y^2, \sqrt{2}xy)$ to a three-dimensional feature space. The points in feature space corresponding to $\mathbf{x}_1$ and $\mathbf{x}_2$ are $\mathbf{x}'_1 = (x_1^2, y_1^2, \sqrt{2}x_1y_1)$ and $\mathbf{x}'_2 = (x_2^2, y_2^2, \sqrt{2}x_2y_2)$. The dot product of these two feature vectors is

$$\mathbf{x}'_1 \cdot \mathbf{x}'_2 = x_1^2x_2^2 + y_1^2y_2^2 + 2x_1y_1x_2y_2 = (x_1x_2 + y_1y_2)^2 = (\mathbf{x}_1 \cdot \mathbf{x}_2)^2.$$

That is, by squaring the dot product in the original space we obtain the dot product in the new space without actually constructing the feature vectors! A function that calculates the dot product in feature space directly from the vectors in the original space is called a **kernel** - here the kernel is $\kappa(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1 \cdot \mathbf{x}_2)^2$.

The mapping function $\phi(\mathbf{x})$ maps the data to a different space. The idea is that learning, say, a linear separator, can be easier in this space.

The kernel function $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j)$ replaces $\mathbf{x}_i \cdot \mathbf{x}_j$.

Algorithm KernelPerceptron(D, η) // perceptron training algorithm using a kernel

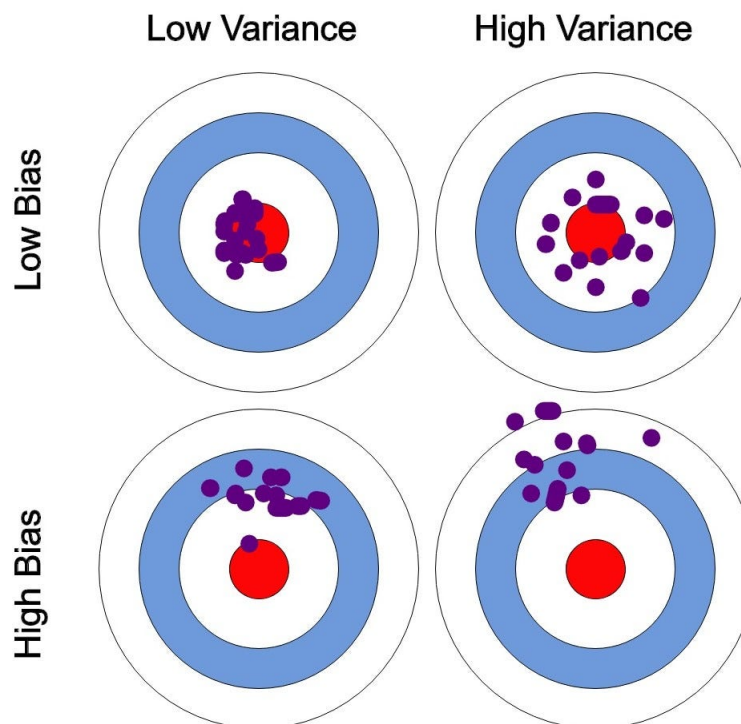
Input: labelled training data D in homogeneous coordinates, plus
kernel function κ

Output: coefficients α_i defining non-linear decision boundary

```
1  $\alpha_i \leftarrow 0$  for  $1 \leq i \leq |D|$ 
2  $converged \leftarrow false$ 
3 while  $converged = false$  do
4    $converged \leftarrow true$ 
5   for  $i = 1$  to  $|D|$  do
6     if  $y_i \sum_{j=1}^{|D|} \alpha_j y_j \kappa(\mathbf{x}_i, \mathbf{x}_j) \leq 0$  then
7        $\alpha_i \leftarrow \alpha_i + 1$ 
8        $converged \leftarrow false$ 
9     end
10  end
11 end
```

5 Ensemble Learning

5.1 Bias and Variance



Bias-variance components of error typically show a trade-off. With increasing bias you get decreasing

variance and vice versa.

5.2 Stability

- for a given data distribution $\mathcal{D}$
- train algorithm L on the training sets S_1, S_2 sampled from $\mathcal{D}$
- expect that the model from L should be the same (or every similar) on both S_1 and S_2
- if so, we say that L is a stable learning algorithm
- otherwise it is unstable
- typical stable algorithm: k NN (for some k)
- typical unstable algorithm: decision-tree learning

Stable algorithms typically have high bias. Unstable algorithms typically have high variance. But take care to consider effect of parameters.

5.3 Ensemble Methods

In essence, ensemble methods in machine learning have the follow two things in common

- they construct multiple, diverse predictive models from adapted versions of the training data (most often reweighted or resampled);
- they combine the predictions of these models in some way, often by simple average or voting (possible weighted).

Basic idea of ensembles learning schemes: build different “experts” and combine their output predictions. Often improved predictive performance but produces output that is very hard to interpret.

5.3.1 Bagging

1. Sample several training sets of size n (instead of just having one training set of size n).
2. Build a classifier for each training set.
3. Combine the classifiers’ predictions.

Reduces variance by voting/averaging, thus reducing the overall expected error.

Algorithm Bagging($D, T, \mathcal{A}$) // train ensemble from bootstrap samples

Input: dataset D ; ensemble size T ; learning algorithm $\mathcal{A}$.

Output: set of models; predictions to be combined by voting or averaging.

```
1 for  $t = 1$  to  $T$  do
2   | bootstrap sample  $D_t$  from  $D$  by sampling  $|D|$  examples with replacement
3   | run  $\mathcal{A}$  on  $D_t$  to produce a model  $M_t$ 
4 end
5 return  $\{M_t | 1 \leq t \leq T\}$ 
```

5.3.2 Random Forest

Like bagging for trees, except that the ensemble of tree models is trained from bootstrap samples and random subspaces.

Algorithm RandomForest(D, T, d) // train ensemble of randomized trees

Input: data set D ; ensemble size T ; subspace dimension d .

Output: set of models; predictions to be combined by voting or averaging.

```
1 for  $t = 1$  to  $T$  do
2   | bootstrap sample  $D_t$  from  $D$  by sampling  $|D|$  examples with replacement
3   | select  $d$  features at random and reduce dimensionality of  $D_t$  accordingly
4   | train a tree model  $M_t$  on  $D_t$  without pruning
5 end
6 return  $\{M_t | 1 \leq t \leq T\}$ 
```

5.3.3 Boosting

Boosting also uses voting/averaging of an ensemble but each model is weighted according to their performance. An iterative learning procedure, new models are influenced by performance of previously built ones.

Algorithm Boosting($D, T, \mathcal{A}$) // train binary classifier ensemble, reweighting datasets

Input: data set D ; ensemble size T ; learning algorithm $\mathcal{A}$

Output: weighted ensemble of models

```
1  $w_{1i} \leftarrow 1/|D|$  for all  $x_i \in D$ 
2 for  $t = 1$  to  $T$  do
3   | run  $\mathcal{A}$  on  $D$  with weights  $w_{ti}$  to produce a model  $M_t$ 
4   | calculate weighted error  $\epsilon_t$ 
5   | if  $\epsilon_t \geq 1/2$  then
6   |   | set  $T \leftarrow t - 1$  and break
7   | end
8   |  $\alpha_t \leftarrow \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$ 
9   |  $w_{(t+1)i} \leftarrow \frac{w_{ti}}{2\epsilon_t}$  for misclassified instances  $x_i \in D$ 
10  |  $w_{(t+1)j} \leftarrow \frac{w_{tj}}{2(1-\epsilon_t)}$  for correctly classified instances  $x_j \in D$ 
11 end
12 return  $M(x) = \sum_{t=1}^T \alpha_t M_t(x)$ 
```

5.3.4 Other Ensemble Learning

Boosting has a more theoretically justified bias and often works better in practice to reduce error, but can be susceptible to very noisy data.

Many other variants of ensemble learning exists. Gradient Boosting (XGBoost, LightGBM, CatBoost) often the “off-the-shelf” method of choice.

6 Neural Learning

6.1 The Multi-layer Perceptron

Perceptrons are linear models with a (discontinuous) activation function, i.e., a threshold on the output. Linear units are differentiable, but do not have a non-linear activation function, so combining multiple linear units is still a linear model.

Sigmoid unit is the same as a perceptron expect that the step function has been replaced by a smoothed version, a sigmoid function.

The sigmoid function has a nice property:

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x)).$$

Notation:

- x_{ji} = the i th input to unit j
- w_{ji} = weight associated with i th input to unit j
- $net_j = \sum_i w_{ji}x_{ji}$ = (weight sum of inputs for unit j)
- o_j = output computed by unit j
- t_j = the target output for unit j
- σ = the sigmoid function
- $outputs$ = the set of units in the final layer of the network
- $Downstream(j)$ = the set of units whose immediate inputs include the output of unit j

6.2 Back-Propagation

Spam chain rule. Back-propagation is “just the chain rule” of calculus

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

Jacobian times gradient

$$\nabla_{\mathbf{x}} z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^{\top} \nabla_{\mathbf{y}} z$$

But it’s a particular implementation of the chain rule on graphs. Uses dynamic programming (table filling), avoids recomputing repeated subexpressions with a speed vs. memory tradeoff.

7 Unsupervised Learning

7.1 Introduction

Supervised learning — classes are known and need a “definition”, in terms of data. Methods are known as: classification, discriminant analysis, class prediction, supervised pattern recognition.

Unsupervised learning – classes are initially unknown and need to be “discovered” with their definitions from the data. Methods are known as: cluster analysis, class discovery, unsupervised pattern recognition.

7.2 Clustering

Finding groups of items that are similar. Success of clustering is often measured subjectively. A dataset for clustering is just like a dataset for classification, but without the class!

Clustering algorithms form two broad categories: hierarchical methods and partitioning methods.

Hierarchical algorithms are either agglomerative (bottom-up) or divisive (top-down). In practice, hierarchical agglomerative methods are often used - efficient exact algorithms available.

Partitioning methods usually require specification of the number of clusters, then try to construct the clusters and fit objects to them.

Let $X = \{e_1, \dots, e_n\}$ be a set of elements i.e. instances. Let $\mathcal{C} = (C_1, \dots, C_l)$ be a partition of X into subsets. Each subset is called a cluster and $\mathcal{C}$ is called a clustering.

Input data can have two forms

- each element is associated with a real-valued vector of p features e.g. measurement levels for different features
- pairwise similarity data between elements, e.g. correlation, distance (dissimilarity)

The goal of clustering is to find a partition of X elements into homogenous and well-separated clusters. Elements from the same cluster should have high similarity.

7.2.1 k -means Clustering

Algorithm to cluster numerical data around k centroids i.e., means. Need to choose value for k , and initialise the centroids.

Find a “good” clustering by iteratively assigning each data point to the nearest of the k centroids, then updating each centroid to reflect any points that have been re-assigned.

$P(i)$ is the cluster assigned to the element i , $c(j)$ is the centroid of cluster j . $d(v_1, v_2)$ the Euclidean distance between feature vectors v_1, v_2 . The goal is to find a partition P for which the error (distance) function $E_P = \sum_{i=1}^n d(i, c(P(i)))$ is minimum. Centroid is the mean or weighted average of points in the cluster.

Algorithm k -means

/* feature-vector matrix $M(ij)$ is given */

- ① Start with an arbitrary partition P of X into k clusters
- ② for each element i and cluster $j \neq P(i)$ let E_P^{ij} be the cost of a solution in which i is moved to j :
 - ① if $E_P^{i^*j^*} = \min_{ij} E_P^{ij} < E_P$ then move i^* to cluster j^* and repeat step 2 else halt.

7.3 Hierarchical Clustering

- Bottom up: at each step join the two closest clusters (starting with single-instance clusters). Design decision: distance between clusters.
- Top down: find two clusters and then proceed recursively for the two subsets. Can be very fast.
- Both methods produce a dendrogram (tree of “clusters”).

Algorithm Hierarchical agglomerative

/* Distance matrix $D(ij)$ is given */

- ① Find minimal entry d_{ij} in D and merge clusters i and j
- ② Update D by deleting column i and row j , and adding new row and column $i \cup j$
- ③ Revise entries in D by recalculating distances between new cluster and existing clusters
- ④ If there is more than one cluster then go to step 1.

Dendrograms. Two things to beware of:

1. tree structure is not unique for given clustering - for each bottom-up merge the sub-tree to the right or left must be specified - 2^{n-1} ways to permute the n leaves in a dendrogram.
2. hierarchical clustering imposes a bias - the clustering forms a dendrogram despite the possible lack of a implicit hierarchical structuring in the data

7.3.1 DBSCAN

Density-based spatial clustering of applications with noise (DBSCAN). Defines clusters as dense regions of data instances. Density is based on having more than a minimum number of points (MinPts) within a radius ϵ . Does not force all points in the dataset to be members of clusters.

- a core point has more than MinPts neighbours within ϵ
- a border point has fewer neighbours than MinPts within ϵ , but is within ϵ of a core point
- all other points are noise points

First, label all points as core, border or noise

- Make a cluster for each core point, or set of connected core points. Core points are connected if they are within ϵ of each other.
- assign each border point to the cluster of the corresponding core point

7.4 Principal Component Analysis

Performs a dimensionality reduction, finding a set of new features which preserves information in the original set.

This algorithm can be presented in several ways:

- eigen-decomposition
- Singular Value Decomposition (SVD)

Can think of PCA as an SVD of the (mean-centred) matrix product $\mathbf{X}^\top \mathbf{X}$.

PCA typically computed using the SVD and complexity is cubic in number of original features. This is not feasible for high-dimensional datasets.

8 Learning Theory

8.1 No Free Lunch Theorem

Assuming that the training set $\mathcal{D}$ can be learned correctly by all algorithms, averaged over all target functions no learning algorithm gives an off-training set error superior to any other:

$$\sum_F [\mathbb{E}_1(E | F, \mathcal{D}) - \mathbb{E}_2(E | F, \mathcal{D})] = 0$$

where F is the set of all possible target functions, E is the off-training set error, and $\mathbb{E}_1, \mathbb{E}_2$ are expectations for two learning algorithms.

8.2 A Conservation Theorem of Generalisation Performance

For every possible learning algorithm for binary classification the sum of performance overall possible target functions is exactly zero!

On some problems we get positive performance, so there must be other problems for which we get an equal and opposite amount of negative performance.

8.3 PAC Learning

There is a whole section on this... Something about Boosting

8.4 Vapnik-Chervonenkis (VC) Dimension

There is also a section on this... Something about SVM

8.5 Mistake Bounds

How many mistakes before convergence?

8.5.1 WINNOW

- WINNOW is an online, mistaken-drive algorithm. Similar to perceptron but converges faster.
- Designed to learn in the presence of many irrelevant attributes. Key improvement: number of mistakes grows only logarithmically with number of irrelevant attributes.
- Key idea: use a multiplicative weight update scheme (as in Boosting).

While some instances are misclassified

For each instance x

 classify x using current weights w

 If predicted class is *incorrect*

 If x has class 1

 For each $x_i = 1$, $w_i \leftarrow \alpha w_i$ # Promotion

 (if $x_i = 0$, leave w_i unchanged)

 Otherwise

 For each $x_i = 1$, $w_i \leftarrow \frac{w_i}{\alpha}$ # Demotion

 (if $x_i = 0$, leave w_i unchanged)

Here x and w are vectors of attributes and weights, respectively.

8.6 The WEIGHTED MAJORITY Algorithm

- learn to predict by weighted vote of set of prediction algorithms
- learns by updating weights for prediction algorithms
- any prediction algorithm simply predicts value of target concept given an instance; a prediction algorithm can be elements of hypothesis space H or even different learning algorithms
- if there is inconsistency between prediction algorithm and training example then reduce weight of prediction algorithm
- bound number of mistakes of ensemble by number of mistakes made by best prediction algorithm

a_i is the i -th prediction algorithm

w_i is the weight associated with a_i

For all i , initialize $w_i \leftarrow 1$

For each training example $(x, c(x))$

Initialize q_0 and q_1 to 0

For each prediction algorithm a_i

If $a_i(x) = 0$ then $q_0 \leftarrow q_0 + w_i$

If $a_i(x) = 1$ then $q_1 \leftarrow q_1 + w_i$

If $q_1 > q_0$ then predict $c(x) = 1$

If $q_0 > q_1$ then predict $c(x) = 0$

If $q_0 = q_1$ then predict 0 or 1 at random for $c(x)$

For each prediction algorithm a_i

If $a_i(x) \neq c(x)$ then $w_i \leftarrow \beta w_i$